

1. Describe the problems that Neural Networks solve.

- a. Neural Networks solve the problem of non-linearity; Neural Networks are able to learn complex patterns where traditional models lack. Neural Networks are also able to learn patterns in high-dimensional data, and learn what humans could not.

2. Explain the difference between supervised, unsupervised, and reinforcement learning.

- a. **Supervised:** Data is labeled. The goal is to minimize the difference between prediction and the actual label.
- b. **Unsupervised:** Data is not labeled; models 'group' data and tries to find patterns without knowing the 'answer'.
- c. **Reinforcement:** The model interacts with some sort of environment and receives rewards/penalties to maximize the total reward over time.

3. RELU (or rectified linear unit) is one type of activation function that is used above. What is the point of having an activation function?

- a. An activation introduces non-linearity. Without an activation function, a Neural Network would essentially be a linear-regression model; with just a stack of linear transforms. Most real-world problems are non-linear, which is why they are so important.

4. How many neurons are in the input layer and what do these neurons correspond to? What about the output layer?

- a. The input layer has one neuron for each feature (column) in the dataset. These input "neurons" don't perform computations - they simply pass the input values into the next layer. Each row in the dataset is one data point fed through these input nodes.
- b. The output layer consists of one or more neurons depending on the task:
 - i. Binary Classification: (One Output)
 - ii. Multiclass Classification: (As many neurons as classes)
 - iii. Regression: (One Neuron)
- c. The output layer receives a weight sum of all neurons in the previous layer to product $\hat{y}$.

5. Explain in your own words the process of back-propagation and how a neural net "learns".

- a. Backpropagation is how a neural network learns. After making a prediction through the forward pass, the network outputs $\hat{y}$. Initially, this prediction may be no better than a guess. The loss function then measures the difference between the predicted value and the true label, giving us the error. Using this error, we compute the partial derivative of the loss (or cost) function with respect to each weight. Once we've derived the general equation for these gradients, we plug in values from the forward pass to calculate the actual gradient for each parameter. These gradients tell us how much each weight contributed to the error. Finally, we use an optimization algorithm like gradient descent to update the weights which nudges them in the direction that reduces the overall cost. This process repeats until the model makes accurate predictions.

6. Is this model a good fit for the data? Why or why not?

- a. It is, because our data is non-linear and is high dimensional; this is what Neural Networks are perfect for.

7. What changes would you make to the Basic_Neural_Networks Notebook to ensure that your group will build a better Neural Network.

- a. Use Pytorch (personal preference)
- b. Perform Feature Engineering
- c. Split into Validation Set (since such low accuracy was achieved to see if model was learning)
- d. Decrease Complexity
- e. Evaluate on other metrics (change loss function)
- f. Experiment with different optimizers as well as a Learning Rate optimizer

8. What is overfitting? How would you prevent overfitting on a Neural Networks

- a. Overfitting is when the model learns the test data too well, essentially memorizing it instead of generalizing to patterns, which is what we want the model to do. This happens often when the model is too complex (too many layers / neurons), or there is not enough training data.

9. What is RMSE and how it is used to evaluate Neural Networks.

- a. RMSE (or Root Mean-Squared Error) is used to determine if our model is actually learning. It can be used as a Cost Function or evaluation metric. It tests the difference between the actual and predicted value and simply square roots the Mean Squared Error (MSE).